UCLA EE 201A -- VLSI Design Automation – Winter 2026
Course Project: Thermal resistance networks
TA: Dedeepyo Ray

******* IMPORTANT *******

- Run your experiments and do your work on SEASnet server `eeapps.seas.ucla.edu`
- You will work in teams of two students.
  - How you coordinate your work is up to you. We recommend a version control system like Git.
  - Be careful not to reveal your project directory to other teams!
- The latest materials are available in GitHub at the link : https://github.com/dedeepyo-eng/ece201a_project/blob/master/ .
- **Due Date:**
  - **Final Code and Report / Slides Submission – 11:59:59 p.m., Wednesday 18th of March**
- Start early! This project will be challenging. There are also expected license and server load issues from this as well as other courses in the last few weeks. You should try and get most work done before that.
- See the end for important submission instructions.

Project Overview

As transistor dimensions shrink and packaging technologies evolve toward complex 2.5D and 3D integrated circuits, power density has skyrocketed, making thermal management a primary gatekeeper of performance in modern VLSI design. For engineers developing high-performance GPUs intended for sustained, long-duration workloads—such as training complex machine learning models or high-fidelity rendering—thermals are no longer an afterthought relegated to the physical packaging phase, but a critical constraint that must be addressed dynamically during early-stage design and architecture. When a GPU runs continuously, it faces severe thermal bottlenecks: concentrated localized hotspots develop within heavily utilized execution units, and the resulting elevated temperatures trigger a vicious cycle of increased leakage power, which in turn generates even more heat.

Furthermore, the modern integration of high-bandwidth memory (HBM) closely stacked with the compute logic introduces massive thermal cross-talk, severely limiting the pathways for heat to escape the package. Without accurate, computationally efficient thermal modeling integrated directly into the design automation flow, these bottlenecks lead to aggressive thermal throttling, accelerate device degradation, and ultimately trap engineers in the "dark silicon" era—where billions of transistors must remain powered down simultaneously simply to prevent catastrophic silicon failure.

While high power densities are a formidable challenge, the advent of 2.5D heterogeneous integration introduces an entirely new problem: catastrophic thermal coupling. In modern systems, we increasingly place high-power compute dies millimeters away from highly temperature-sensitive components, most notably High Bandwidth Memory (HBM) stacks. Because HBM relies on capacitive DRAM cells, elevated temperatures increase capacitor leakage, which forces the memory to drastically increase its refresh

rates to prevent data loss. This thermal throttling can crater effective memory bandwidth by up to 27% and inflate read latency by 70%, severely bottlenecking memory-bound machine learning workloads.

Historically, engineers tried to solve this with thermal-aware floorplanning—simply spacing the chiplets further apart on the silicon interposer. However, in a 2.5D package, the primary conduit for inter-chiplet heat transfer is actually the highly conductive copper heat spreader sitting on top of the dies, not the interposer below. Because this heat spreader acts as a lateral thermal superhighway, spacing chiplets far enough apart to meaningfully reduce temperatures requires impractical interposer area increases of up to 55%. This completely defeats the purpose of tight integration, degrading communication efficiency and massively inflating manufacturing costs.

This critical bottleneck motivates a targeted architectural intervention: embedded thermal isolators. By physically embedding ultra-low thermal conductivity materials—such as aerogel, air gaps, or FR-4—directly into the copper heat spreader between the chiplets, designers can effectively sever the lateral heat pathways and partition the package into distinct thermal zones. This targeted decoupling approach can successfully drop HBM temperatures below critical throttling thresholds without expanding the interposer footprint, reclaiming up to 37% of lost system performance in memory-bound AI applications.

In the current project, we will use a system description model to specify a single GPU + 6 HBMs large system and extract a thermal resistance network out of it. The latter will then need to be fed to a resistance solver (PiSPICE) to extract the temperature map.

You will be given multiple combinations of GPU + HBM chiplets and cooling solutions. The Figure of Merit is the variation in temperature between your RC solver + PySPICE and our golden result.

Update : Power consumption of the Power_Source is set to 0 for ease. The GitHub code has been updated, lines 13, 1597, 1770, 1771 have been commented out.

Notes:

- *It is recommended that you adopt version control such as Git for collaborating with your teammate. If you are not comfortable with Git, you can read Mark's tutorial at https://markgottscho.wordpress.com/2015/01/24/a-brief-git-tutorial-for-collaborative-research/ . We also recommend beginners start with www.gitimmersion.com .*
- *Please use Piazza for asking (and answering) questions before coming to office hours.*
- *Since you are expected to work on the assignment in teams of two, do not provide solutions or compare results with other teams.*

Project Files

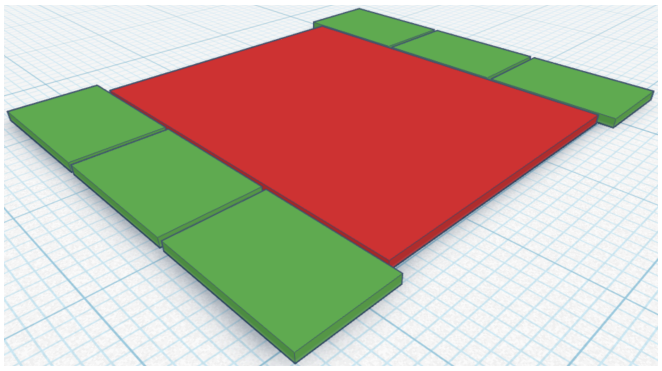
You will be given code that takes a system description model config and creates a 3D stackup of boxes. You need to add code to create a 3D grid of thermal resistances out of the stackup of chiplets and the cooling solution on top.

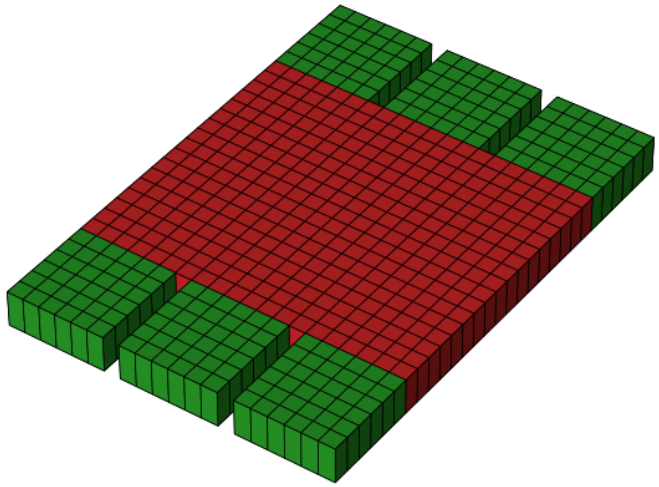
One way to solve this is to divide the entire 3D system into small, uniform boxes and calculate the

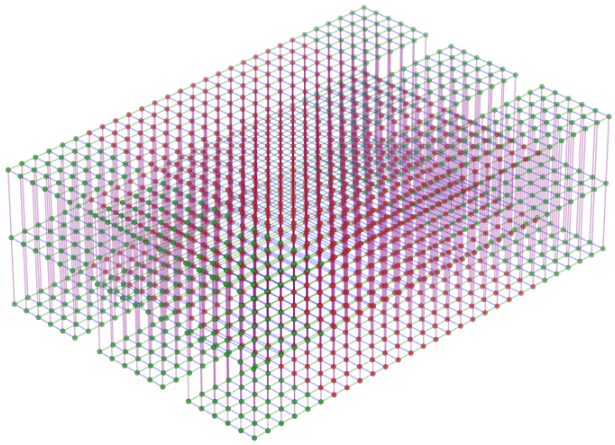
thermal resistance (along X, Y and Z) of each small box in your grid. You are free to play around with the granularity of your grid. Another step further could be implementing non-uniform grid sizes since the chiplets are not all of the same size.

The 3D grid should be fed as an input to PySPICE (version : https://github.com/PySpice-org/PySpice) to extract the temperature map of the system and get the peak temperature of the GPU and HBM systems. The power consumed by the GPU should be estimated to be 270 W and each HBM should be assumed to consume 5 W of power.

Example :: The below stackup can be divided into the below mentioned grid structure which gives the below resistive network.







Starter Code

You are given a Python program therm.py which calls a function simulator_simulate() which you need to define. The simulator_simulate() function takes the set of boxes as input. Each box is an instance of Box() class, having start_x, start_y, start_z, end_x, end_y, end_z coordinates and stackup. The stackup consists of a list of layers, each of which has a material compositions and thickness. The definition of all layers is also passed as an argument to simulator_simulate() function. All heatsinks are defined in the heatsink_definitions.xml and the heatsink we are using is specified using heatsink_water_cooled. You are allowed to pass more inputs as arguments to the simulator_simulate() function if required.

The code is run using below commands for the three starter configurations.

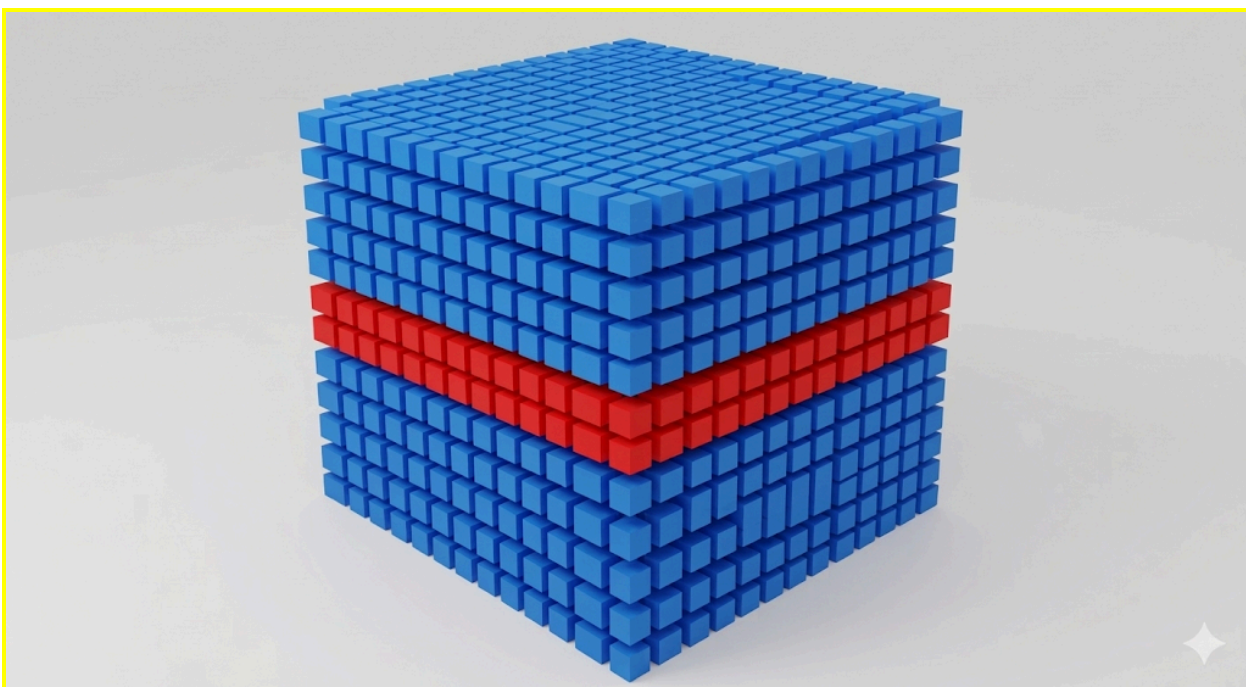
```
python3 therm.py --therm_conf
/app/nanocad/projects/deepflow_thermal/DeepFlow/configs/thermal-configs/sip_hbm_dray_0
62325_1GPU_6HBM_3D_single_GPU_on_top.xml --out_dir out_therm --heatsink_conf
/app/nanocad/projects/deepflow_thermal/DeepFlow/configs/thermal-configs/heatsink_defin
itions.xml --bonding_conf
/app/nanocad/projects/deepflow_thermal/DeepFlow/configs/thermal-configs/bonding_defini
tions.xml --heatsink heatsink_water_cooled --project_name
ECTC_3D_1GPU_8high_120125_higherHTC --is_repeat False --hbm_stack_height 8
--system_type 3D_1GPU_top --dummy_si True --tim_cond_list 5 --infill_cond_list 1.6
--underfill_cond_list 1.6
```

```
python3 therm.py --therm_conf
/app/nanocad/projects/deepflow_thermal/DeepFlow/configs/thermal-configs/sip_hbm_dray_0
62325_1GPU_6HBM_3D_single_GPU.xml --out_dir out_therm --heatsink_conf
/app/nanocad/projects/deepflow_thermal/DeepFlow/configs/thermal-configs/heatsink_defin
itions.xml --bonding_conf
/app/nanocad/projects/deepflow_thermal/DeepFlow/configs/thermal-configs/bonding_defini
tions.xml --heatsink heatsink_water_cooled --project_name
ECTC_3D_1GPU_8high_110325_higherHTC --is_repeat False --hbm_stack_height 8
--system_type 3D_1GPU --dummy_si True --tim_cond_list 5 --infill_cond_list 1.6
--underfill_cond_list 1.6
```

```
python3 therm.py --therm_conf
/app/nanocad/projects/deepflow_thermal/DeepFlow/configs/thermal-configs/sip_hbm_dray06
2325_1gpu_6hbm_2p5D.xml --out_dir out_therm --heatsink_conf
/app/nanocad/projects/deepflow_thermal/DeepFlow/configs/thermal-configs/heatsink_defin
itions.xml --bonding_conf
/app/nanocad/projects/deepflow_thermal/DeepFlow/configs/thermal-configs/bonding_defini
tions.xml --heatsink heatsink_water_cooled --project_name
ECTC_2p5D_1GPU_8high_110325_higherHTC --is_repeat False --hbm_stack_height 8
--system_type 2p5D_1GPU --dummy_si False --tim_cond_list 5 --infill_cond_list 1.6
--underfill_cond_list 1.6
```

Sample output for the second command line has been attached in Piazza.

The power of the GPU is distributed in a plane at the middle of the vertical cross-section. One way to model this is to uniformly distribute power in all the grid cells in the vertical centre plane of the GPU, as in the image attached below. Same for each tier in the HBM. Red voxels emit GPU power and blue voxels don't.



The simulator_simulate() function is expected to return a dictionary as below.

results = simulator_simulate()
results = {
“Box1” : (peak_temperature_of_box1, average_temperature_of_box1, thermal_resistance_of_box1_in_x, thermal_resistance_of_box1_in_y, thermal_resistance_of_box1_in_z),

"Box2" : (peak_temperature_of_box1, average_temperature_of_box1, thermal_resistance_of_box1_in_x, thermal_resistance_of_box1_in_y, thermal_resistance_of_box1_in_z),

...

...

...

}

A few important points are mentioned below.

1. Ambient temperature is to be mentioned in report. Take the ambient temperature to be 45 C.
2. All temperatures must be reported in degrees Celsius.
3. Reports / slides must mention the approach used, whether uniform or non-uniform meshing was used, how voxel resistances were calculated.
4. Grading will be based on correctness and runtime. For correctness, focus will be on how close the variance of the original results matches with the NGSpice results, in addition to how far the correct results are from the NGSpice results

Project Submission

We require each group to submit their code as a single therm.py file. Submit any additional python files that your code references. In addition, each group is also required to submit a report / slides explaining their methods / algorithms and mentioning their results.

Be absolutely sure that you can run the complete flow without problems. Otherwise, your submission cannot be graded!

SUBMISSION INSTRUCTIONS (Read carefully)

- All necessary code must be tarballed and submitted as a single archive file on `eeapps.seas.ucla.edu`. Presentation slides (PowerPoint or PDF format) should be posted on bruinlearn by the deadline.
  - For final code submission, create a directory named as follows: `GroupID_Lastname1-Firstname1_UID1_username1_Lastname2-Firstname2_UID2_username2_Project/` . For example: `Group1_Bruin_Joe_123456789_joe_Bruin_Josie_987654321_josie_Project/`
    - Include all files mentioned in the Project Submission section.
    - Compress and archive this directory using tar/gzip to have a single submission file named `GroupID_Lastname1-Firstname1_UID1_username1_Lastname2-Firstname2_UID2_username2_Project_pinXXXX.tar.gz` (Make up a 4-digit numeric PIN of your choice to substitute for `XXXX`). For example: `Group1_Bruin_Joe_123456789_joe_Bruin_Josie_987654321_josie_Project_pin1234.tar.gz`
    - Submit tarball by copying it to `/w/class.1/ee/ee201o/ee201ot2/submission/project/`

    - IMPORTANT: Make sure all files you submit, as well as the tarball, have full read and execute permissions to group and others or we may not be able to grade your lab. Do this using `chmod -R go+rx FILE_OR_DIRECTORY`
- **Late submissions will not be accepted** (write/execute permissions to the submission directory will be removed). If you cannot finish the entire assignment on time, submit whatever you completed. **No submission = no points.** You are welcome to overwrite your submission as many times as you like before the deadline.
- Some students may worry that their work could be visible to other students. We try to reduce this chance by disabling read permissions on `/w/class.1/ee/ee201o/ee201ot2/2024_labs/project/` directory so that other students cannot list its contents. Thus, they will not know the filename and cannot open your submission unless you give them your full name, student ID number, SEAS username, and arbitrary 4-digit PIN that you substituted for `xxxx` earlier on the tarball. This also means you will not be able to list the directory contents to see if your own submission worked – you can only write to it! In short, please do not give your classmates this information or collaborate on homeworks. The PIN can be whatever 4-digit number you want -- it is just to reduce the likelihood of another student guessing your full file submission path. You can change it for every lab submission if you like.
- **Test your submission before the deadline!**